

Software Maturity Engine: Pre-Launch Revenue, ROI, and Commercialization Analysis

Alexander, the commercial thesis is unusually strong **provided the product is positioned as a software-maturation system rather than another autonomous developer**. The relevant customer purchase is not “write code for me”; it is “use otherwise uneconomic machine effort to discover and close residual software risk before customers, regulators, incidents, and production traffic do it for us.”

The kernel is technically aligned with that proposition: it separates planning, authorization, mutation, independent oracle verification, and closure; forbids self-grading; requires layer-correct falsifiable evidence; tracks unresolved residuals; and only permits `SUCCESS` after acceptance coverage, an oracle pass, and materially different from a coding agent whose success criterion is task completion.

Executive summary

Inferred — investment thesis: the best initial category is **pre-launch computational software maturation**, sold as a quality/reliability assurance layer on top of existing CI, testing, security, and coding-agent stacks. This avoids direct “replace engineers” positioning and attacks a workload that human teams routinely underperform economically: repeated exploration of edge cases, recovery paths, migrations, configuration states, stale artifacts, integration boundaries, and false-positive test closure.

The commercial opportunity is large enough without inventing a new TAM. Grand View Research estimates the global software-testing market at **\$52.7 billion in 2026**, growing to \$93.2 billion by 2033; managed testing is currently the largest service model, automation the largest testing method, and AI-augmented/autonomous testing the fastest-growing method. North America represented 35.3% of the market in 2025.

¹ Application security is an adjacent \$12.6 billion 2026 market, but I would **not add it to TAM** because the overlap would produce double-counting. ²

The demand-side economics are favorable. Uptime Institute reports that **57% of respondents whose organizations had a major outage said their most recent major outage cost over \$100,000, and one in five said an impactful outage exceeded \$1 million**. ³ IBM's 2026 breach study reports a **\$4.99 million global average data-breach cost**, with extensive security AI/automation associated with \$1.93 million lower average breach cost. ⁴ Those figures do not mean every bug is worth hundreds of thousands of dollars; they establish that preventing even a small number of high-consequence escapes can economically justify six-figure annual software-maturity spend.

The direct inference cost is surprisingly small relative to that value. At current OpenAI enterprise token rates, a representative mixed-model agent call under the assumptions detailed below costs about **\$0.031 of model inference**, or roughly **\$3.11 per 100 calls**. OpenAI currently prices GPT-5.6 Sol at \$4/\$0.40/\$20 per million uncached-input/cached-input/output tokens, Terra at \$2/\$0.20/\$12, and Luna at \$0.20/\$0.02/\$1.20.

⁵ At Alexander's internal endurance datum of 800 calls over eight hours, this implies model inference

measured in tens of dollars rather than thousands; the economically dominant costs are likely to be enterprise integration, human adjudication, customer success, security/compliance, and sales—not tokens.

Recommended commercial structure: start with a paid **Maturity Sprint at \$25,000–\$50,000**, then convert successful pilots into **\$120,000–\$250,000 annual contracts**, with regulated/private-environment deployments typically **\$250,000–\$600,000+**, plus metered overage. Do not initially sell pure outcome-based contracts; attribution of “defects that would have escaped” is too contentious.

Under the explicitly modeled **Likely** scenario—40/90/160 paid pilots in Years 1–3, \$30,000 pilot price, 40% pilot-to-annual conversion, \$150,000 annual contract value, 90% renewal, 78% gross margin, and \$1.5 million initial productization investment—the model produces:

Metric	Modeled result
Year 1 revenue	\$2.40M
Year 2 revenue	\$7.56M
Year 3 revenue	\$16.40M
Cumulative three-year revenue	\$26.36M
Three-year gross profit	\$20.56M
Three-year operating cash flow before initial investment	\$8.06M
NPV at 12%	\$4.29M
Modeled IRR	76%
Investment payback	~2.03 years , early Year 3

These are **Exact arithmetic on Hypothetical commercial assumptions**, not forecasts of realized sales.

The most important sensitivity result is that **sales conversion and realized ACV matter far more than model-compute cost**. In the Likely model, three-year NPV falls below zero around a **23.3% pilot-to-contract conversion rate** or an **\$87.6k annual ACV**. By contrast, modeled direct compute expense can rise about **6.4×** before NPV reaches zero, assuming other COGS are unchanged. The product therefore needs to optimize proof of customer value before it optimizes token spend.

My overall recommendation is **GO to a controlled commercial pilot phase, not yet GO to broad launch**. The launch gate should be empirical: at least five lighthouse pilots, zero critical false closures, 100% intention-preservation, $\geq 95\%$ rejection of deliberately seeded false-PASS traps, median MGC $\geq 5\times$, and $\geq 30\%$ paid pilot-to-annual conversion.

Market size, customer segments, and willingness to pay

TAM, SAM, and SOM

The cleanest TAM is existing quality spend rather than hypothetical labor replacement.

Layer	Calculation	Market size	Interpretation
Core TAM	2026 worldwide software-testing market	\$52.7B	Existing budget pool already purchasing reliability/quality work. ¹
Adjacent pool	2026 application-security market	\$12.6B	Important cross-sell/ROI category, but excluded from TAM addition to avoid overlap. ²
Economic value pool	US cost of poor software quality, 2022	\$2.41T	Economic ceiling, not a market-size estimate ; CISQ also estimated ~\$1.52T of accumulated technical debt. ⁶
North-American testing spend	\$52.7B × 35.3%	\$18.60B	Geography grounded in GVR's reported regional share. ¹
Initial SAM	\$18.60B × assumed 45% suitability	\$8.37B	Inferred filter for pre-launch, automation-accessible, high-complexity enterprise/ISV workloads.
Likely Year-3 SOM	\$16.404M / \$8.371B	0.196% of SAM	Sales-capacity-based SOM rather than arbitrary market-share target.
Optimistic Year-3 SOM	\$66.945M / \$8.371B	0.80% of SAM	Still below 1% of the modeled beachhead market.

The **45% SAM filter is deliberately an assumption**, not a market-research statistic. Define it as:

$$SAM = TAM_{NA} \times f_{accessible}$$

where $f_{accessible}$ represents organizations with software valuable enough to justify repeated maturation, accessible code/runtime/test environments, and procurement tolerance for autonomous pre-production tooling. The model uses $f_{accessible} = 0.45$; pilot data should replace this assumption.

This sizing is conservative in one important sense: it does not add application security, SRE/observability, incident-prevention services, technical-debt remediation, or regulatory assurance as independent TAMs. Those are upsell surfaces, not additive markets.

Best customer segments

Inferred: regulated industries and infrastructure-intensive software should be targeted before generic SaaS.

Segment	Why maturity has economic value	Initial WTP hypothesis	Best entry product
Enterprise software organizations	Large integration surfaces, migrations, legacy behavior, expensive releases	\$120k–\$300k ACV	Continuous Maturity
Independent software vendors	Product reputation and customer-support costs concentrate risk into releases	\$75k–\$250k ACV	Pre-release Maturity Sprint → annual
Financial services / fintech	Operational-resilience obligations plus high incident exposure	\$250k–\$600k+ ACV	Regulated Assurance
Medical-device / health technology	Formal quality assurance and auditable software evidence	\$200k–\$500k+ ACV	Regulated Assurance
Cloud, database, networking, infra vendors	Rare state combinations and failures have unusually high downstream cost	\$150k–\$500k ACV	Reliability/Maturity Engine
AI infrastructure / agent platforms	Rapidly changing behavior, nonlinear integration risk, growing generated-code volume	\$150k–\$500k ACV	Continuous Maturity + agent-specific probes

These price bands are **Inferred commercial hypotheses**, not observed market prices.

There is, however, strong objective evidence for high willingness to spend on risk reduction. DORA requires covered financial entities to establish and maintain a comprehensive digital operational-resilience testing program intended to identify weaknesses, deficiencies, and gaps and to implement corrective measures; Article 25 explicitly enumerates vulnerability analysis, source-code review where possible, scenario testing, compatibility, performance, end-to-end, and penetration testing. ⁷

For medical-device production and quality-management software, FDA's February 2026 Computer Software Assurance guidance explicitly advocates a **risk-based approach to establishing confidence** and recognizes different testing and assurance activities. FDA's revised Quality Management System Regulation became effective on February 2, 2026. ⁸

NIST similarly frames secure software development as continuous work to reduce vulnerabilities, mitigate the impact of undetected defects, and prevent recurrence; its 2026 DevSecOps work explicitly emphasizes automation and standardization of these practices. ⁹

These are excellent entry markets because the maturity engine can sell **evidence** rather than merely code generation.

Unit economics and pricing architecture

Cost per agent call and compute-hour

A useful cost model is:

$$C_{call} = \frac{T_u P_i + T_c P_c + T_o P_o}{10^6} + C_{tools}$$

where T_u is uncached input, T_c cached input, T_o output, and P_i, P_c, P_o are the respective token rates.

For planning, I use a representative call envelope of:

20,000 *input*, 80% *cached*, 2,000 *output*

This is an **assumption**, not a measured average from the kernel.

At current OpenAI enterprise rates: ¹⁰

Model	Representative token cost/call
GPT-5.6 Sol	\$0.0624
GPT-5.6 Terra	\$0.0352
GPT-5.6 Luna	\$0.00352
GPT-5.3-Codex	\$0.0378

A routing mix of 20% Sol, 50% Terra, and 30% Luna yields:

$$C_{call} \approx \$0.03114$$

Using Alexander's **internal benchmark** of approximately 100 calls/hour:

$$C_{inference/hour} \approx 100 \times 0.03114 = \$3.11$$

As a public cloud-development infrastructure benchmark, GitHub currently charges \$0.36/hour for a four-core Codespace and \$0.18/hour for two cores. ¹¹

A deliberately conservative planning multiplier of 2.5× for CI jobs, browsers, storage, network, retries, specialized test environments, and other tool execution gives:

$$C_{fully\ loaded\ direct\ compute/hour} \approx (\$3.11 + \$0.36) \times 2.5 = \boxed{\$8.68}$$

Thus an eight-hour/800-call run would have a modeled direct infrastructure cost around:

$$8 \times \$8.68 \approx \boxed{\$69}$$

A prudent launch budget should nevertheless assume **\$10–\$30 per active maturity-agent hour**, not \$8.68, until actual telemetry is collected. Device farms, GPU workloads, proprietary test environments, large-context calls, external APIs, and model-price changes can dominate individual jobs.

Anthropic's current Sonnet 5 list price is \$2/million input and \$10/million output, further supporting the conclusion that frontier-model inference itself is no longer necessarily the dominant COGS item for enterprise coding workflows. ¹²

Value per discovered defect

Avoid the old and poorly defensible “a production bug always costs 100× more” heuristic. The correct metric is expected avoided loss:

$$V_i = P_{escape,i} [P_{incident|escape,i} L_{incident,i} + \Delta C_{remediation,i}] \times A_i$$

where A_i is an attribution/conservatism factor.

Examples illustrate the economics:

Escape scenario	Loss anchor	Incident probability	Expected avoided value
Defect could trigger a \$100k outage	\$100k	5%	\$5,000
Defect could trigger a \$1M outage	\$1M	5%	\$50,000
Security defect contributes to average global breach	\$4.99M	1%	\$49,900
Same breach-class defect at 5% probability	\$4.99M	5%	\$249,500

The \$100k/\$1M outage anchors come from Uptime's 2026 analysis; the \$4.99M breach anchor is IBM's 2026 global mean. ¹³ The probabilities are modeling assumptions and should be agreed with each customer's reliability/security team.

This produces a much more defensible sales argument: **a \$30k pilot does not need to find hundreds of bugs**. At a conservative average expected value of \$7,500 per independently validated escaped-defect-equivalent, four such findings offset a \$30k pilot fee before considering engineer time saved.

Maturity metering

I recommend separating a **billing unit** from a **quality unit**.

A **Maturity Compute Unit (MCU)** can initially equal:

$$1MCU = 100 \text{ normalized agent/tool calls}$$

with model-tier weights applied so that expensive calls consume proportionately more units.

At 100 calls/hour this is approximately one active agent-hour in Alexander's current benchmark. The base modeled direct compute cost is roughly \$8.7/MCU.

Quality should instead be reported through oracle-stamped residual closure, ERR, FCRR, and other maturity KPIs. **Never sell “number of findings” as the primary unit**, because it creates the same perverse incentive that billable-hour consulting and alert-count security products suffer from.

Recommended pricing

Package	Proposed price	Scope	Commercial purpose
Maturity Sprint	\$25k-\$50k/project	One launch candidate, fixed scope, baseline + long-run maturation + evidence report	Low-friction paid proof
Continuous Maturity	\$120k-\$200k/year	Multiple repositories/releases, scheduled runs, approximately 450-750 MCU included	Core ISV/enterprise product
Enterprise Assurance	\$250k-\$600k+/year	Private runners, SSO/RBAC, audit retention, multiple teams/repos, 1,500+ MCU	Enterprise platform
Regulated Assurance add-on	+\$75k-\$200k/year	Evidence retention, approval workflow, regulatory mappings, controlled change	Finance/medical/critical systems
MCU overage	\$150-\$250/MCU initially	Metered excess compute	Budget predictability
Outcome bonus	5-10% of independently validated avoided value, capped	Optional after baseline data exists	Aligns incentives without making revenue un-auditable

Inferred recommendation: the best default is **annual platform minimum + included MCU + overage**, not pure compute pricing. Cognition independently validates the acceptability of this pattern: Devin Enterprise is metered in Agent Compute Units tied to the amount and complexity of agent work, VM use, network, browser use, planning, and code execution. ¹⁴

At the other extreme, pure subscription hides cost and makes heavy users unattractive. Pure project pricing prevents expansion. Pure outcome pricing creates attribution fights. The hybrid captures all four advantages.

Competitive landscape and defensibility

The category is **not empty**. Several vendors have moved materially toward agentic quality and pre-incident reliability during 2026. The positioning must therefore be sharper than “AI that tests software.”

Offering	Current public position	Pricing signal	Overlap with Maturity Engine	Publicly visible gap relative to kernel
Tricentis	Enterprise agentic quality engineering; AI-agent orchestration, governance, human oversight	Enterprise sales	High	Strongest direct QA competitor. Public positioning emphasizes quality engineering/testing rather than repeated epistemic fixed-point convergence. ¹⁵
Gremlin / Foresight AI	AI agents find, remediate, test, and verify reliability risks	Quote; platform priced per service, Foresight per team	Very high in resilience	Closest conceptual competitor; but centered on operational resilience/failure modes rather than whole-project maturation. ¹⁶
mabl	Agentic web/mobile/API/AI testing, autonomous coverage, runtime recovery, failure triage	Custom enterprise pricing, cloud-test credits	High in application testing	Excellent autonomous test maintenance; public product scope remains testing rather than claim-ledger/oracle closure across arbitrary software layers. ¹⁷
BrowserStack	Real-device/browser testing plus AI/agentic test generation and repair	Self-service from tens/hundreds monthly; enterprise bundles	Medium	Execution infrastructure and test generation rather than autonomous maturation. ¹⁸
OpenAI Codex / Agents API	General-purpose long-running code agents, subagents, managed sandboxes, persistent intermediate state	Token/credit based	High enabling-platform threat	Horizontal agent substrate, not currently publicly framed as fixed-point software-maturity assurance. Agents API is explicitly designed for agents running reliably for days. ¹⁹
Anthropic Claude Code / Sonnet	General agentic coding with strong coding/reasoning models	Token and subscription usage	Medium/high enabling-platform threat	General software agent, not a dedicated maturity/independent-oracle product. ¹²

Offering	Current public position	Pricing signal	Overlap with Maturity Engine	Publicly visible gap relative to kernel
Cognition Devin	Autonomous software-engineering tasks with enterprise ACU metering	ACU usage under enterprise contracts	Medium/high	Optimizes autonomous work completion; public billing itself shows the viability of compute-normalized agent work. 20
Traditional QA/SRE services	Human/automation-led testing, reliability and managed services	Project/FTE/managed-service contracts	High budget overlap	Expensive to apply recursively hundreds of times; slower marginal cycle cost

The critical conclusion is that **“agentic testing” is already competitive**. Tricentis launched what it describes as an end-to-end enterprise agentic quality-engineering platform in March 2026. 21 Gremlin goes further and publicly says its Foresight agents can scan and test systems for potential failures, produce concrete remediation, and verify resilience. 16

Therefore the kernel's moat cannot be:

AI writes tests, finds bugs, and fixes them.

That moat is disappearing.

The defensible proposition is:

Convergence under independent falsification

The kernel's differentiating protocol is that a plausible result from an adjacent layer does not complete the claim, model confidence is not evidence, the mutation role does not silently become the verification role, and completion depends on independent reproducible oracle evidence.

This creates four potential product moats:

Epistemic provenance. Every material claim has state, falsifier, evidence and a layer-correct oracle rather than a conversational “done.”

Anti-tunnel perturbation. Independent alternative decomposition—your “friend's call”—does not vote on truth; sufficiently large divergence causes the hypothesis space to reopen and the full proof cycle to run again.

Maturity convergence. The stopping criterion can evolve from “task finished” into “independent perturbation ceases to discover material residual.”

Auditability. This is commercially important in regulated sectors. A trace of claims, evidence, residual risk, scope and reproducible oracle results is materially more procurement-friendly than opaque model confidence.

OpenAI's new Agents API materially lowers implementation risk because it now explicitly supports managed long-running agents that maintain files, code, intermediate results, tools, and subagents over multi-day workloads. ²² It also creates platform risk: if this category proves valuable, horizontal agent providers can move upward. The moat therefore needs to reside in **protocol, evaluation corpus, maturity telemetry, customer-specific oracle integrations, and accumulated defect/convergence data**, not in model access.

Go-to-market, adoption barriers, and pilot design

Adoption barriers

The hardest problem is unlikely to be compute.

Trust and false closure are barrier one. A system that silently “certifies” its own repairs is unacceptable for high-value customers. The kernel's authority separation is therefore a product feature, not an internal implementation detail.

Code/data sovereignty is barrier two. Enterprises will need private repositories, secrets, logs, test data, production-like environments and potentially regulated records exposed to the maturation process. Enterprise packaging should therefore support customer-controlled runners/VPCs, BYOM/BYO-model where required, strict read/write scopes, immutable evidence logs, SSO/RBAC, and configurable retention.

Unbounded agent spend is barrier three. Current vendors already expose usage controls because autonomous work is inherently variable: Devin meters ACUs and GitHub provides spend controls for metered development environments. ²³ The kernel should expose hard MCU ceilings per run, repository, team and month.

Finding noise is barrier four. Buyers will not tolerate a \$250k product that creates another backlog of speculative AI defects. Findings should be commercially counted only after an independent reproducible oracle or explicit human adjudication.

Organizational resistance is barrier five. Positioning matters. Sell:

“Perform the reliability work your team cannot economically perform exhaustively before launch.”

not:

“Replace your QA/SRE/development organization.”

Recommended sales motion

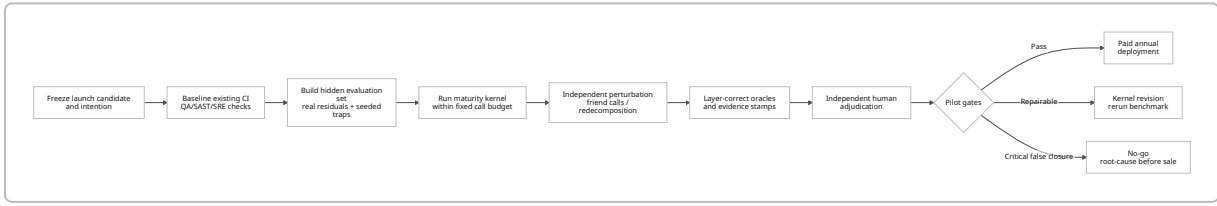
The initial motion should be **founder/technical sales into lighthouse projects with obvious downside risk**, followed by land-and-expand.

The first target should not be companies optimizing a low-risk CRUD application. Prioritize releases involving migrations, distributed systems, stateful recovery, compatibility, safety, financial flows, critical infrastructure, complex desktop/industrial software, or significant customer deployment cost.

Gremlin's own packaging is evidence that customers accept a staircase from expert-led baseline engagement to ongoing reliability platform and then agentic automation. Its current offering explicitly includes a Reliability Baseline, peak-readiness engagements, continuous reliability roadmaps and an AI agent add-on. ²⁴ That is a useful commercial analogue.

Pilot experiment

A defensible pilot should compare **incremental verified maturity**, not raw number of bugs.



The evaluation set should contain both genuine unknown defects and **deliberate adversarial traps**. In particular, seed situations where parser tests pass but persisted state is wrong; unit tests pass but runtime behavior is wrong; source state is correct but generated artifacts are stale; mocked interfaces pass but integration fails; and visual/UI claims pass type-checking but not rendered observation.

That directly tests the kernel's unique claim rather than generic coding ability.

Pilot KPI contract

Define a severity-weighted residual R before the run:

$$R = w_A A + w_C C + w_R R_c + w_U U$$

where A = uncovered acceptance items, C = unstamped claims, R_c = critical risks, and U = unresolved residual. Weights must be pre-registered so the agent cannot improve its score by redefining importance after seeing results.

Then:

$$ERR = 1 - \frac{R_{final}}{R_{initial}}$$

$$MGC = \frac{\sum_i V_i}{C_{compute} + C_{human-validation} + C_{integration}}$$

$$TRE = \frac{N_{\text{stalls producing new verified geometry}}}{N_{\text{detected stalls}}}$$

$$FCRR = \frac{N_{\text{seeded/plausible false closures rejected}}}{N_{\text{false-closure traps presented}}}$$

$$IPR = 1 - \frac{N_{\text{unauthorized target mutations}}}{N_{\text{runs}}}$$

I recommend these **Inferred launch thresholds**:

KPI	Pilot threshold	Hard/soft
ERR	≥85% reduction in pre-registered weighted residual	Soft commercial gate
MGC	≥5.0× median across pilots	Commercial gate
TRE	≥80% stalled episodes escape to materially new, subsequently verified geometry	Technical gate
Stall recovery cost	p95 ≤100 additional calls to new verified medoid	Efficiency target
FCRR	≥95% overall	Hard
Critical false-PASS traps	100% rejected	Hard
IPR	100.0%	Hard
Critical regressions introduced	0	Hard
Fixed-point stability	3 independent re-decomposition cycles , each ≤2% change in weighted residual and no new Sev-1/Sev-2 finding	Maturity gate
Paid-pilot conversion	≥30% after first 5–10 pilots	Business gate

The fixed-point requirement is crucial. A single converged run can simply be a stable wrong model. Repeated convergence **after independent perturbation** is a substantially stronger test.

Financial scenarios and sensitivity

Model assumptions and formulas

All scenario projections below are **Hypothetical inputs with Exact arithmetic**.

Annual revenue is modeled as:

$$Revenue_y = P_y P_{pilot} + ACV (Renewed_y + 0.5 \times Converted_y)$$

with:

$$Converted_y = P_y \times Conversion$$

$$Renewed_y = Active_{y-1} \times Renewal$$

New conversions receive half a year of recognized recurring revenue to approximate pilots occurring throughout the year.

Gross profit:

$$GP_y = Revenue_y \times GM$$

Operating cash flow:

$$FCF_y = GP_y - OPEX_y$$

and:

$$NPV = -I_0 + \sum_{y=1}^3 \frac{FCF_y}{(1+r)^y}$$

where base discount rate $r = 12\%$ and assumed pre-launch productization investment $I_0 = \$1.5M$.

Variable	Conservative	Likely	Optimistic
Paid pilots Y1/Y2/Y3	24 / 50 / 80	40 / 90 / 160	60 / 160 / 320
Pilot price	\$20k	\$30k	\$40k
Pilot → annual conversion	25%	40%	55%
Annual ACV	\$75k	\$150k	\$275k
Renewal	85%	90%	92%
Gross margin	65%	78%	84%
OPEX Y1/Y2/Y3	\$2.2/\$2.8/\$3.6M	\$2.5/\$4.0/\$6.0M	\$3.0/\$7.5/\$15.0M
Initial investment	\$1.5M	\$1.5M	\$1.5M

Three-year financial outcome

Scenario	Y1	Y2	Y3	3Y revenue	3Y COGS	3Y OPEX	Gross margin
Conservative	\$0.705M	\$1.851M	\$3.472M	\$6.028M	\$2.110M	\$8.600M	65%
Likely	\$2.400M	\$7.560M	\$16.404M	\$26.364M	\$5.800M	\$12.500M	78%
Optimistic	\$6.938M	\$26.849M	\$66.945M	\$100.732M	\$16.117M	\$25.500M	84%

Return metrics			
Scenario	NPV @12%	IRR	Payback
Conservative	-\$5.284M	N/M	>3 years
Likely	\$4.288M	76.1%	2.03 years
Optimistic	\$42.374M	~400%	0.53 years

N/M means no meaningful positive IRR exists because the modeled cash flows remain negative throughout the three-year horizon.

Modeled revenue trajectories

The asymmetry is notable. The Conservative model fails badly; this is **not** a business where a weak pilot funnel can be compensated by cheap AI. The Likely case crosses viability largely because paid pilots convert into six-figure recurring contracts.

Sensitivity

For the price-per-maturity-unit sensitivity, assume:

$$ACV = \$30,000 \text{ platform minimum} + 600 \times Price_{MCU}$$

Thus the Likely \$150k ACV corresponds to a \$200 MCU price.

For compute sensitivity, the base 22% COGS assumption is decomposed as:

$$COGS = 18\% \text{ other variable COGS} + 4\% \text{ compute}$$

and only the compute component is multiplied.

Variable	Low	Base	High	NPV low	NPV base	NPV high
MCU price	\$100	\$200	\$300	\$0.17M	\$4.29M	\$8.41M
Pilot conversion	20%	40%	60%	-\$0.86M	\$4.29M	\$9.44M
Average ACV	\$75k	\$150k	\$250k	-\$0.86M	\$4.29M	\$11.15M
Compute cost	0.5×	1×	4×	\$4.68M	\$4.29M	\$1.91M

NPV sensitivity

The approximate break-even points under this model are:

$$Conversion_{BE} \approx \boxed{23.3\%}$$

$$ACV_{BE} \approx \boxed{\$87.6k}$$

$$Price_{MCU, BE} \approx \boxed{\$96}$$

with the assumed \$30k platform minimum and 600 MCU annual use.

Direct compute can rise approximately:

$$\boxed{6.4\times}$$

before three-year NPV reaches zero, corresponding to a gross margin near 56%, if the rest of the model remains unchanged.

This leads to a strong strategic conclusion:

commercial proof and ACV $\gg$ token optimization

for the first phase of the business.

That is especially plausible because long-horizon agent infrastructure is rapidly becoming commoditized. OpenAI's September 10, 2026 Agents API explicitly provides hosted Codex infrastructure for agents operating reliably over days, with tool use, subagent orchestration and persistent intermediate state. ²² OpenAI also reports that by May 2026 more than 70% of Codex users had requested work estimated to take a human more than one hour, while its heaviest internal users were generating more than 60 hours of agent turns per day across parallel agents by June. ²⁵ Long-running compute is therefore increasingly available; **what the compute is instructed to converge on** is the differentiated asset.

Recommended launch strategy, go/no-go criteria, and risks

The product should be named and sold around **maturity, release assurance, or pre-production resilience**, not autonomous coding.

The core economic proposition should be:

Spend machine time before launch to obtain failure history without waiting years for customers and production to create it.

More precisely, the product converts calendar-time maturation into compute:

Raw software -> Explore -> Falsify -> Repair -> Verify
Perturb -> Repeat -> Robust epistemic fixed point

That positioning is reinforced by NIST research showing that systematic test design can dramatically compress otherwise exhaustive search: its combinatorial-testing program reports multiple studies achieving equal fault detection to exhaustive testing with **20×–700× smaller test sets**. The Maturity Engine is not itself combinatorial testing, but this is strong evidence for the broader engineering principle that structured exploration can convert an intractable state space into tractable computational assurance. ²⁶

Commercial sequence

First 90 days: recruit 5–10 lighthouse customers rather than launching broadly. Target one infrastructure vendor, one enterprise SaaS/ISV, one fintech/financial system, one regulated health/medical software environment, and one complex stateful/industrial application. Charge for every pilot; a free pilot destroys the very willingness-to-pay signal being tested.

Months 3–6: convert successful pilots to annual contracts, harden evidence reports, private runner architecture, spend controls and organization-wide maturity dashboards. Do not expand model features until customer evidence shows where value accrues.

Months 6–12: establish benchmark datasets consisting of real historical escaped defects, seeded wrong-layer oracle traps, recovery failures, stale artifacts, migrations and compatibility defects. This corpus becomes part of the moat.

Year 2: add partner/OEM routes into testing consultancies, regulated assurance practices, agent platforms and cloud marketplaces. OpenAI/Anthropic-like horizontal agent providers are simultaneously compute suppliers, distribution partners and potential future competitors.

Go/no-go

I would use a three-level decision rather than one aggregate score.

GO only if, across at least five paid pilots, the median ERR is $\geq 85\%$, median MGC $\geq 5\times$, FCRR $\geq 95\%$, critical false-closure rejection is 100%, IPR is 100%, no critical regression is introduced, at least three independent perturbation/re-decomposition passes converge without new high-severity residual, and $\geq 30\%$ of pilots sign an annual contract.

REPAIR AND RETEST if technical reliability passes but conversion is below 30%, or commercial value is demonstrated but ERR/MGC misses threshold. Those indicate packaging/segment/efficiency problems rather than an invalid product thesis.

NO-GO until root cause is eliminated after **any known critical false closure**, unauthorized target mutation, fabricated oracle evidence, systematic self-grading path, or a result in which the system reports maturity while independent validation finds a material uncovered acceptance condition. Those failures attack the product's central proposition rather than a peripheral feature.

The most important risk mitigation is therefore not “make the model smarter.” It is preserving the kernel's epistemic constraints as non-negotiable product invariants.

A maturity report should never claim:

"The software is correct."

It should say:

"Within these declared surfaces, environments, acceptance conditions and perturbations, these claims have reproducible evidence; these residuals remain Unknown; further independent passes produced less than ϵ new weighted residual."

That distinction is commercially valuable. It is also what prevents "Software Maturity" from becoming an indefensible certification claim.

Bottom-line recommendation

Inferred: GO to monetized pilots.

The market is sufficiently large, the direct-compute economics are unusually favorable, regulatory and operational-risk buyers already spend heavily on adjacent assurance, and horizontal agents now provide enough long-running infrastructure to make the architecture operationally plausible. ²⁷

The likely business case is attractive **only if the system reliably converts pilots into six-figure annual contracts**. The financial model says the critical business numbers are approximately:

$$ACV > \$88k, \quad conversion > 23\%, \quad target\ conversion \geq 30\% - 40\%$$

The recommended commercial targets are therefore:

$$\$30k\ pilot \rightarrow \$150k\ median\ ACV \rightarrow 40\%\ conversion \rightarrow 78\% + gross\ margin$$

and the recommended technical product promise is:

$$\text{verified residual reduction under repeated independent perturbation}$$

rather than number of tests, number of fixes, amount of generated code, or number of agent-hours.

That is the key strategic distinction. **The Software Maturity Engine does not compete primarily for developer labor. It competes for the enormous amount of economically valuable verification and maturation work that organizations currently perform incompletely because human attention is too expensive.** Existing software-testing spend gives it a credible budget pool; incident economics give it ROI headroom; long-running agent infrastructure makes the cost base practical; and the kernel's independent-oracle design gives it a potentially defensible reason to exist beyond general coding agents. ²⁸

Keywords: software-maturity 0.23, revenue 0.18, ROI 0.17, oracle-convergence 0.15, enterprise-pricing 0.11, pilot-KPI 0.09, go-to-market 0.07
Semantic dominant: Commercialize the kernel as a pre-launch computational

maturity engine whose value is verified residual reduction under independent perturbation, with success determined by paid conversion and evidence-backed customer ROI rather than code-generation volume.

md5: 4b9d7e2a81cf4635b0a736d9e5241f8c

prev-md5: e81c5a0d741a4bd89d37c64f12ae905b

parent-goal-md5: 00000000000000000000000000000000

-
- 1 27 28 <https://www.grandviewresearch.com/industry-analysis/software-testing-market-report>
<https://www.grandviewresearch.com/industry-analysis/software-testing-market-report>
 - 2 <https://www.grandviewresearch.com/industry-analysis/application-security-market>
<https://www.grandviewresearch.com/industry-analysis/application-security-market>
 - 3 <https://uptimeinstitute.com/about-ui/press-releases/uptime-announces-annual-outage-analysis-report-2026>
<https://uptimeinstitute.com/about-ui/press-releases/uptime-announces-annual-outage-analysis-report-2026>
 - 4 <https://newsroom.ibm.com/2026-07-29-ibm-study-one-in-four-malicious-breaches-are-ai-enabled%2C-costing-companies-6-million-on-average?lnk=hpln1id>
<https://newsroom.ibm.com/2026-07-29-ibm-study-one-in-four-malicious-breaches-are-ai-enabled%2C-costing-companies-6-million-on-average?lnk=hpln1id>
 - 5 10 <https://help.openai.com/en/articles/20001415-chatgpt-rate-card-enterprise-token-based-pricing>
<https://help.openai.com/en/articles/20001415-chatgpt-rate-card-enterprise-token-based-pricing>
 - 6 <https://www.it-cisq.org/press-releases/12-06-22/>
<https://www.it-cisq.org/press-releases/12-06-22/>
 - 7 <https://eur-lex.europa.eu/legal-content/FR-EN/TXT/?uri=CELEX%3A32022R2554>
<https://eur-lex.europa.eu/legal-content/FR-EN/TXT/?uri=CELEX%3A32022R2554>
 - 8 <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/computer-software-assurance-production-and-quality-management-system-software>
<https://www.fda.gov/regulatory-information/search-fda-guidance-documents/computer-software-assurance-production-and-quality-management-system-software>
 - 9 <https://www.nist.gov/news-events/news/2025/12/secure-software-development-framework-ssdf-version-12-available-public>
<https://www.nist.gov/news-events/news/2025/12/secure-software-development-framework-ssdf-version-12-available-public>
 - 11 <https://github.com/pricing/calculator>
<https://github.com/pricing/calculator>
 - 12 <https://www.anthropic.com/news/claude-sonnet-5>
<https://www.anthropic.com/news/claude-sonnet-5>
 - 13 <https://intelligence.uptimeinstitute.com/index.php/resource/annual-outage-analysis-2026>
<https://intelligence.uptimeinstitute.com/index.php/resource/annual-outage-analysis-2026>
 - 14 20 23 <https://docs.devin.ai/admin/billing>
<https://docs.devin.ai/admin/billing>

15 21 <https://www.tricentis.com/news/tricentis-introduces-agentic-software-quality-platform>
<https://www.tricentis.com/news/tricentis-introduces-agentic-software-quality-platform>

16 24 <https://www.gremlin.com/pricing>
<https://www.gremlin.com/pricing>

17 <https://www.mabl.com/pricing>
<https://www.mabl.com/pricing>

18 <https://www.browserstack.com/pricing>
<https://www.browserstack.com/pricing>

19 22 <https://openai.com/index/introducing-the-agents-api/>
<https://openai.com/index/introducing-the-agents-api/>

25 <https://openai.com/index/how-agents-are-transforming-work/>
<https://openai.com/index/how-agents-are-transforming-work/>

26 <https://www.nist.gov/programs-projects/combinatorial-testing>
<https://www.nist.gov/programs-projects/combinatorial-testing>